



OBJECT ORIENTED SYSTEMS (OOS) — 2025

QUESTION 1 COMPLETE DETAILED SOLUTION



Introduction

Question 1 of the 2025 Object Oriented Systems examination is one of the most important sections of the paper because it thoroughly evaluates the conceptual foundation of Java programming and Object-Oriented Programming principles 🚀🍷. This question belongs to CO1 and is entirely focused on:

- Java core concepts
- Runtime behavior
- Object-oriented reasoning
- Inheritance & Polymorphism
- Interfaces & Arrays
- Constructors
- Packages
- String handling

Unlike ordinary syntax-based examinations, this paper tests whether the student truly understands:

- **WHY** Java behaves in a certain way,
- **HOW** the JVM internally manages execution,
- and **WHAT** design principles are followed behind the language.

The concepts discussed in this question are extremely important because they form the foundation of enterprise Java development, multithreaded systems, framework development, Android applications, backend architecture, and advanced software engineering 🍷.

A student who deeply understands these concepts can easily transition into advanced areas like reflection, generic programming, design patterns, distributed systems, and scalable software development 🚀.



QUESTION 1(a)(i)

? “There is only one entry point to a program, irrespective of the number of `main()` methods in a class.”

? Justify the validity of the statement with supporting arguments and suitable code snippets.

✅ Answer

The statement is completely valid because in Java, the execution of every standalone application begins from only one specific method whose signature is predefined by the Java Virtual Machine (JVM). That official entry point is:

```
public static void main(String args[])
```

Although Java supports **method overloading**, including overloading of the `main()` method itself, the JVM recognizes only the above method signature as the starting point of program execution 🚀. Therefore, even if

multiple versions of `main()` are written inside the same class, only one method acts as the true program entry point.

🌟 Detailed Explanation

The JVM requires a standard mechanism to start program execution. Without a fixed entry point, ambiguity would arise regarding which method should execute first, how command-line arguments should be passed, and how the runtime environment should initialize the application ⚠️. To solve this problem, Java enforces one standardized entry method.

💻 Code Example

```
class Demo {  
    public static void main(String args[]) {  
        System.out.println("Original main() method");  
        main(10);  
        main("Java");  
    }  
  
    public static void main(int x) {  
        System.out.println("Integer overloaded main(): " + x);  
    }  
  
    public static void main(String s) {  
        System.out.println("String overloaded main(): " + s);  
    }  
}
```

💻 Output

```
Original main() method  
Integer overloaded main(): 10  
String overloaded main(): Java
```

🌟 Deep Conceptual Analysis

When the JVM loads the class, it searches specifically for `public static void main(String args[])` and invokes it automatically.

The other overloaded versions (`main(int x)` and `main(String s)`) are treated like ordinary methods. They are executed only because they are explicitly called from inside the original `main()` method. Thus, overloaded `main()` methods are not separate program entry points.

Internally, the JVM checks the access modifier, return type, parameter type, and static nature before deciding which method qualifies as the official startup method 🧠.

🌟 Real-world Intuition

Consider a university examination hall 🏫. Even though multiple doors may exist inside the building, students are allowed to enter only through the officially designated entrance gate. Similarly, multiple `main()` methods may exist, but the JVM always begins execution through one official entry point.

🎯 Conclusion

Although Java allows multiple overloaded versions of `main()`, the JVM always starts execution from only one predefined method. Hence a Java application always has exactly one true entry point 🚀.

📄 QUESTION 1(a)(ii)

? **“Method overriding is also known as late binding.”**

? **Justify the validity of the statement with supporting arguments and suitable code snippets.**

✅ Answer

The statement is absolutely valid because method overriding supports **runtime polymorphism**, where the actual method to execute is determined dynamically during runtime rather than during compilation 🚀.

This runtime method resolution process is called 🔥 **Late Binding** or 🔥 **Dynamic Binding** because the association between the method call and actual method implementation occurs late, that is, during execution time.

💡 Detailed Explanation

Method overriding occurs when a child class defines a method having the same name, same parameter list, and compatible return type as a method already present in the parent class. The child class then provides its own specialized behavior.

💻 Code Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

class Main {
    public static void main(String args[]) {
        Animal a = new Dog();
        a.sound();
    }
}
```

Output

Dog barks

🌟 Deep Conceptual Analysis

The reference variable `Animal a` belongs to the parent type. However, the actual object created is `new Dog()`. Thus during runtime, the JVM examines the real object type and invokes `Dog.sound()` instead of `Animal.sound()`. This decision is postponed until execution time. Therefore, overriding is called late binding 🗨️.

🌟 JVM Internal Working

Internally the JVM maintains virtual method tables, runtime object metadata, and dynamic dispatch mechanisms. When overridden methods are invoked, the JVM dynamically determines the correct method implementation according to the actual object type. This provides flexibility and extensibility in software systems 🚀.

🌟 Real-world Intuition

Suppose a payment application supports UPI, Credit Card, and Net Banking. All payment systems may implement the same method `pay()`, but actual payment behavior depends on the object selected during runtime. This is late binding in real-world software architecture 💡.

🎯 Conclusion

Method overriding supports runtime polymorphism where method resolution occurs during execution time. Hence overriding is also known as late binding or dynamic binding 🚀.

QUESTION 1(a)(iii)

? **“A non-static member method can be invoked within the static block of a class.”**

? **Justify the validity of the statement with supporting arguments and suitable code snippets.**

✅ Answer

The statement is conditionally valid because a non-static member method cannot be called directly inside a static block, but it can be invoked indirectly using an object reference 🚀.

🌟 Detailed Explanation

Static blocks belong to the **class level**, whereas non-static methods belong to the **object level**. Therefore, non-static methods require object memory, an object reference, and instance context. Since static blocks execute before object creation, direct invocation becomes illegal ⚠️.

❌ Invalid Program

```
class Demo {  
    void show() {  
        System.out.println("Hello");  
    }  
  
    static {  
        show(); // ERROR  
    }  
}
```

Output

Compiler Error:
Non-static method cannot be referenced from static context

Correct Program

```
class Demo {  
    void show() {  
        System.out.println("Non-static method invoked");  
    }  
  
    static {  
        Demo d = new Demo();  
        d.show();  
    }  
  
    public static void main(String args[]) {  
    }  
}
```

Output

Non-static method invoked

Deep Conceptual Analysis

Static blocks execute during class loading. At that moment no object exists automatically. Therefore, the JVM cannot determine which object's method should execute. By explicitly creating an object (`Demo d = new Demo();`), object context becomes available. Then non-static method invocation becomes possible.

Conclusion

A non-static method cannot be directly invoked inside a static block, but it can be called through object creation. Hence the statement is conditionally valid 🚀.

QUESTION 1(a)(iv)

? **“Reference data types do not store the value directly in a variable.”**

? **Justify the validity of the statement with supporting arguments and suitable code snippets.**

✅ Answer

The statement is valid because reference variables in Java do not directly store actual object data. Instead, they store memory references, object addresses, or object locations that point to objects stored in heap memory 🚀.

🌟 Detailed Explanation

Java variables are broadly categorized into primitive datatypes and reference datatypes. Primitive variables directly store actual values (e.g., `int x = 10;` directly stores the value 10). However, reference variables behave differently.

Code Example

```
class Student {  
    int roll = 5;  
}  
  
class Main {  
    public static void main(String args[]) {  
        Student s = new Student();  
        System.out.println(s.roll);  
    }  
}
```

Output

5

🌟 Deep Conceptual Analysis

The variable `Student s` does not contain the entire object. Instead, the actual object is stored in the heap, and the variable stores a reference pointing toward the heap object.

Memory representation: `s` -----> `Student Object in Heap`

🌟 Why Java Uses References

Reference-based memory management provides dynamic object creation, polymorphism, garbage collection, memory efficiency, and object sharing. These are essential for modern object-oriented systems 🚀.

🎯 Conclusion

Reference datatypes store references to heap objects rather than storing actual object data directly inside variables. Hence the statement is valid 🚀.

📄 QUESTION 1(a)(v)

? **"Every member method in a Base class can be overridden in its Child class."**

? **Justify the validity of the statement with supporting arguments and suitable code snippets.**

✅ Answer

The statement is invalid because not all methods in a parent class are eligible for overriding ⚠️. Only inheritable non-final instance methods can be overridden.

Methods that cannot be overridden include:

- **final** methods
- **private** methods
- **static** methods
- Constructors

💡 Detailed Explanation

Method overriding exists to provide runtime polymorphism. However, certain methods are intentionally protected from overriding to preserve security, consistency, and language design integrity.

💻 Code Example

```
class Base {  
    final void show() {  
        System.out.println("Base Method");  
    }  
}  
  
class Child extends Base {  
    void show() {  
        System.out.println("Child Method");  
    }  
}
```

💻 Output

Compiler Error:

Cannot override final method

💡 Why certain methods cannot be overridden

- **final Methods:** The keyword `final` means implementation must remain unchanged. Child classes cannot modify behavior.
- **private Methods:** Private methods are not inherited. Since the child class cannot access them, overriding becomes impossible.
- **static Methods:** Static methods belong to the class itself. They undergo *method hiding*, not overriding.

🎯 Conclusion

Only inheritable non-final instance methods can participate in overriding. Therefore the statement is invalid



📄 QUESTION 1(a)(vi)

? **"An interface solves the diamond problem automatically."**

? **Justify the validity of the statement with supporting arguments and suitable code snippets.**

✅ Answer

The statement is valid because Java interfaces eliminate the ambiguity associated with multiple inheritance and thereby solve the diamond problem safely 🚀.

💡 Detailed Explanation

The diamond problem occurs when multiple parent classes contain the same method, and a child inherits from all of them. This creates ambiguity regarding which method implementation should execute ⚠️.

Java avoids this issue because multiple inheritance of classes is prohibited. However, interfaces safely support multiple inheritance.

💻 Code Example

```
interface A {  
    void show();  
}  
  
interface B extends A {}  
interface C extends A {}  
  
class D implements B, C {  
    public void show() {  
        System.out.println("Diamond problem resolved");  
    }  
}
```

Output

Diamond problem resolved

🌟 Deep Conceptual Analysis

Interfaces traditionally contain abstract methods. Thus the implementing class must explicitly define the implementation. Since the implementation is provided directly by the child class, ambiguity disappears. Therefore, the diamond problem gets resolved automatically.

🎯 Conclusion

Interfaces eliminate ambiguity by forcing explicit implementation inside child classes. Hence interfaces solve the diamond problem safely 🚀.

QUESTION 1(a)(vii)

? “The `ensureCapacity()` method doubles the capacity of a `StringBuffer` object.”

? Justify the validity of the statement with supporting arguments and suitable code snippets.

✅ Answer

The statement is valid because the `ensureCapacity()` method of the **`StringBuffer`** class is internally designed to dynamically increase the storage capacity whenever the existing character buffer becomes insufficient 🚀.

In Java, `StringBuffer` is a mutable sequence of characters. Unlike the `String` class, whose objects are immutable, `StringBuffer` objects can be modified repeatedly without creating new objects each time. Because of this mutability, Java must internally manage memory expansion efficiently so that repeated append operations do not degrade performance ⚠️.

When the current capacity of a `StringBuffer` becomes insufficient, Java automatically expands its internal storage. The expansion mechanism generally follows the formula: **New Capacity = (Old Capacity × 2) + 2**

🌟 Detailed Explanation

Whenever a `StringBuffer` object is created without specifying capacity explicitly, Java allocates a default capacity of 16 characters. As more characters are inserted, the JVM checks whether enough free space is available. If additional space is required, a larger memory block is allocated, old data is copied, and capacity expands dynamically.

Code Example

```
class Main {  
    public static void main(String args[]) {  
        StringBuffer sb = new StringBuffer();  
        System.out.println("Initial Capacity = " + sb.capacity());  
    }  
}
```

```
        sb.append("abcdefghijklmnopq");
        System.out.println("New Capacity = " + sb.capacity());
    }
}
```

Output

```
Initial Capacity = 16
New Capacity = 34
```

Deep Conceptual Analysis

Initially, the `StringBuffer` object has a default capacity of 16. However, the appended string contains 17 characters. Since the current capacity becomes insufficient, Java internally applies the resizing formula: $(16 \times 2) + 2 = 34$.

Example Using `ensureCapacity()`

```
class Main {
    public static void main(String args[]) {
        StringBuffer sb = new StringBuffer();
        System.out.println(sb.capacity());

        sb.ensureCapacity(50);
        System.out.println(sb.capacity());
    }
}
```

Output

```
16
50
```

Why Capacity Becomes 50: The method `ensureCapacity(50)` requests a minimum capacity of 50. Since 50 exceeds the automatically calculated doubled size (34), Java directly allocates a 50-character capacity.

Conclusion

The `ensureCapacity()` method dynamically expands the internal storage of `StringBuffer` objects, generally following a doubling strategy $(oldCapacity \times 2) + 2$, improving performance and memory efficiency 🚀.

QUESTION 1(a)(viii)

? "Two strings can be compared by == operator."

? Justify the validity of the statement with supporting arguments and suitable code snippets.

✓ Answer

The statement is conditionally valid because the `==` operator in Java compares **references**, not actual string contents ⚠️. Therefore two strings can indeed be compared using `==`, but this comparison checks whether both references point to the same memory object rather than checking whether the textual contents are equal.

🌟 Detailed Explanation

In Java, strings are objects stored in heap memory or the string constant pool. The `==` operator checks whether both references refer to identical object locations. It does NOT compare character sequences. Content comparison should normally use the `equals()` method.

💻 Code Example 1 — Reference Comparison

```
class Main {
    public static void main(String args[]) {
        String s1 = "Java";
        String s2 = "Java";
        System.out.println(s1 == s2);
    }
}
```

💻 Output

true

Why Output is true: String literals are stored inside the String Constant Pool. Since both literals are identical, the JVM reuses the same object.

💻 Code Example 2 — Different Objects

```
class Main {
    public static void main(String args[]) {
        String s1 = new String("Java");
        String s2 = new String("Java");

        System.out.println(s1 == s2);
        System.out.println(s1.equals(s2));
    }
}
```

💻 Output

```
false  
true
```

🌟 Deep Conceptual Analysis

Here, `new String("Java")` creates separate heap objects. Thus, the references differ even if the contents remain the same. Therefore `==` returns false, while `equals()` returns true.

🎯 Conclusion

Two strings can technically be compared using `==`, but the operator compares references rather than textual contents. Content comparison should normally use `equals()` 🚀.

📄 QUESTION 1(a)(ix)

- ? **"A private member method in a Base class cannot be overridden in its Child class."**
? **Justify the validity of the statement with supporting arguments and suitable code snippets.**

✅ Answer

The statement is completely valid because private methods are not inherited by child classes 🚀. Method overriding requires inheritance, accessibility, and visibility. Since private methods are accessible only inside their own class, child classes cannot access or inherit them.

🌟 Detailed Explanation

The keyword `private` restricts method visibility strictly to the same class only. Thus child classes cannot even see private methods. If a child class defines a method with the same signature, it becomes an entirely new method, not an override.

💻 Code Example

```
class Base {  
    private void show() {  
        System.out.println("Base Method");  
    }  
}  
  
class Child extends Base {  
    void show() {  
        System.out.println("Child Method");  
    }  
}  
  
class Main {  
    public static void main(String args[]) {  
        Child c = new Child();  
        c.show();  
    }  
}
```

```
}  
}
```

Output

Child Method

Deep Conceptual Analysis

The method `private void show()` belongs exclusively to the `Base` class. It is invisible outside `Base`. Thus `Child.show()` does not override the parent method; it defines a completely separate method.

Conclusion

Private methods are not inherited and therefore cannot participate in method overriding. Hence the statement is completely valid 🚀.

QUESTION 1(a)(x)

? **"Two different packages should never have classes having same name."**

? **Justify the validity of the statement with supporting arguments and suitable code snippets.**

Answer

The statement is invalid because Java packages are specifically designed to avoid naming conflicts 🚀. Different packages CAN contain classes having identical names because package names provide unique namespaces.

Detailed Explanation

Packages provide modularity, namespace management, access control, and conflict prevention. Thus two different packages may safely contain classes with the same class name without ambiguity.

Code Example

Package p1

```
package p1;  
public class Demo {  
    public void show() {  
        System.out.println("Package p1");  
    }  
}
```

Package p2

```
package p2;
public class Demo {
    public void show() {
        System.out.println("Package p2");
    }
}
```

■ Main Program

```
import p1.*;
import p2.*;

class Main {
    public static void main(String args[]) {
        p1.Demo d1 = new p1.Demo();
        p2.Demo d2 = new p2.Demo();

        d1.show();
        d2.show();
    }
}
```

🖥 Output

```
Package p1
Package p2
```

🌟 Deep Conceptual Analysis

Although both classes have the same name (**Demo**), their fully qualified names differ (**p1.Demo** and **p2.Demo**). Thus the JVM treats them as completely separate classes.

🎯 Conclusion

Different packages can safely contain classes with identical names because packages provide unique namespaces. Therefore the statement is invalid 🚀.

📄 QUESTION 1(b)(i)

? "The difference between the keywords **final** and **finally** is ____."

? Fill in the blank and justify the validity of the completed sentence with proper arguments and suitable code snippets.

✅ Answer

🔥 “The difference between the keywords **final** and **finally** is that **final** is used to restrict modification whereas **finally** is used for exception handling cleanup.”

🌟 Detailed Explanation

Although the words appear similar syntactically, they belong to completely different areas of Java programming 🚀. **final** is associated with restriction, immutability, and inheritance control. On the other hand, **finally** belongs to exception handling and is used to execute cleanup code regardless of whether an exception occurs or not ⚠️.

💻 Example of final Variable

```
class Main {  
    public static void main(String args[]) {  
        final int x = 10;  
        // x = 20; // Reassignment causes compiler error  
        System.out.println(x);  
    }  
}
```

💻 Example of finally

```
class Main {  
    public static void main(String args[]) {  
        try {  
            int x = 10 / 0;  
        } catch (Exception e) {  
            System.out.println("Exception Occurred");  
        } finally {  
            System.out.println("Finally Block Executed");  
        }  
    }  
}
```

🌟 Comparison Table

Feature	final	finally
Type	Keyword	Block
Purpose	Restriction	Cleanup
Used In	Variables, methods, classes	Exception handling
Inheritance Control	Yes	No

🎯 Conclusion

The keyword `final` restricts modification and inheritance, whereas `finally` ensures execution of cleanup code during exception handling 🚀.

QUESTION 1(b)(ii)

? "Classes in a package must be declared as _____, otherwise they cannot be _____."

✓ Answer

🔥 "Classes in a package must be declared as `public`, otherwise they cannot be accessed outside the package."

🌟 Detailed Explanation

If a class inside a package is not declared as `public`, its visibility becomes package-private by default. This means it can only be accessed inside the same package, not from outside packages ⚠️.

Code Example

```
// Package p1
package p1;
class Demo {
    void show() { System.out.println("Hello"); }
}

// Another Package
import p1.*;
class Main {
    public static void main(String args[]) {
        Demo d = new Demo(); // Compiler Error: Demo is not public in p1
    }
}
```

🎯 Conclusion

Classes inside packages must be declared `public` if they need to be accessed outside their package 🚀.

QUESTION 1(b)(iii)

? "Static methods in a Base class can be overridden, but it results in method _____."

✓ Answer

🔥 "Static methods in a Base class can be overridden, but it results in method hiding."

🌟 Detailed Explanation

Static methods belong to the class level, not the object level. Therefore they do not participate in runtime polymorphism, dynamic dispatch, or true overriding. When a child class defines a static method having the same signature as a parent static method, the child method hides the parent method instead of overriding it 🚀.



Code Example

```
class Base {
    static void show() { System.out.println("Base Method"); }
}

class Child extends Base {
    static void show() { System.out.println("Child Method"); }
}

class Main {
    public static void main(String args[]) {
        Base b = new Child();
        b.show();
    }
}
```

Output

Base Method

Conclusion

Static methods do not support true overriding. Child static methods merely hide parent static methods 🚀.

QUESTION 1(b)(iv)

? "A _____ array is called a jagged array if _____."

✓ Answer

🔥 "A multidimensional array is called a jagged array if its rows contain different number of columns."

Detailed Explanation

In Java, multidimensional arrays are implemented as arrays of arrays. Thus each row can independently store different column sizes. When rows contain unequal column lengths, the array becomes a jagged array, ragged array, or irregular array 🚀.

Code Example

```
class Main {  
    public static void main(String args[]) {  
        int arr[][] = {  
            {1, 2},  
            {3, 4, 5},  
            {6}  
        };  
  
        for(int i = 0; i < arr.length; i++) {  
            for(int j = 0; j < arr[i].length; j++) {  
                System.out.print(arr[i][j] + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
1 2  
3 4 5  
6
```

Conclusion

A multidimensional array having unequal row sizes is called a jagged array 🚀.

QUESTION 1(b)(v)

? "An abstract class may not have an _____, but it must be _____."

✓ Answer

🔥 "An abstract class may not have an abstract method, but it must be declared abstract."

💡 Detailed Explanation

An abstract class is a class that cannot be instantiated directly 🚀. A class may be declared abstract even without abstract methods. But if any method inside a class is abstract, then the class itself must be declared abstract ⚠️.

Code Example

```
abstract class Demo {  
    void show() {  
        System.out.println("Concrete Method");  
    }  
}
```

```
}  
}
```

Conclusion

An abstract class may contain only concrete methods, but if a class is declared abstract it cannot be instantiated directly 🚀.

QUESTION 1(b)(vi)

? "Run time polymorphism occurs only for _____ of a class, but not for _____."

✓ Answer

🔥 "Run time polymorphism occurs only for methods of a class, but not for data members."

💡 Detailed Explanation

Runtime polymorphism in Java is achieved using method overriding. The JVM dynamically decides which overridden method executes based on the actual object type 🚀. However, data members do not participate in runtime polymorphism. Variable access depends upon the reference type, not the object type ⚠️.

Code Example

```
class Base {  
    int x = 10;  
    void show() { System.out.println("Base Method"); }  
}  
  
class Child extends Base {  
    int x = 20;  
    void show() { System.out.println("Child Method"); }  
}  
  
class Main {  
    public static void main(String args[]) {  
        Base b = new Child();  
        System.out.println(b.x);  
        b.show();  
    }  
}
```

Output

```
10  
Child Method
```

Conclusion

Runtime polymorphism applies only to overridden methods, not data members 🚀.

QUESTION 1(b)(vii)

? “The difference between checked and unchecked exception is _____.”

? Fill in the blank and justify the validity of the completed sentence with proper arguments and suitable code snippets.

✅ Answer

🔥 “The difference between checked and unchecked exception is that checked exceptions are checked during compilation whereas unchecked exceptions are checked during runtime.”

🌟 Detailed Explanation

Java provides a powerful exception handling mechanism to manage abnormal situations during program execution 🚀.

- **Checked Exceptions** are verified by the compiler during compilation. The programmer is forced to handle them using `try-catch` or declare them using `throws`. They generally occur due to external conditions (like file handling or network issues). Examples: `IOException`, `SQLException`.
- **Unchecked Exceptions** occur during runtime and are not checked by the compiler ⚠️. They mainly happen due to logical programming mistakes. Examples: `ArithmeticException`, `NullPointerException`.

Code Example — Checked Exception

```
import java.io.*;

class Main {
    public static void main(String args[]) throws IOException {
        FileReader fr = new FileReader("abc.txt");
    }
}
```

Code Example — Unchecked Exception

```
class Main {
    public static void main(String args[]) {
        int x = 10 / 0;
        System.out.println(x);
    }
}
```

🌟 Comparison Table

Feature	Checked Exception	Unchecked Exception
Checked By	Compiler	JVM at Runtime
Handling Mandatory	Yes	No
Cause	External conditions	Programming mistakes
Examples	<code>IOException</code>	<code>ArithmeticException</code>

Conclusion

Checked exceptions are verified during compilation and must be handled explicitly, whereas unchecked exceptions occur during runtime due to programming errors .

QUESTION 1(b)(viii)

? "The parameterized constructor of the Base class must be called from _____ by _____."

✓ Answer

🔥 "The parameterized constructor of the Base class must be called from the Child class constructor by using `super()`."

🌟 Detailed Explanation

In Java inheritance, constructors are not inherited directly . However, whenever a child object is created, the parent constructor executes first. If the parent class contains only a parameterized constructor, then the child constructor must explicitly invoke it using `super(parameters)`. Otherwise, a compilation error occurs .

Code Example

```
class Base {
    Base(int x) {
        System.out.println("Base Constructor: " + x);
    }
}

class Child extends Base {
    Child() {
        super(100);
        System.out.println("Child Constructor");
    }
}

class Main {
    public static void main(String args[]) {
        Child c = new Child();
    }
}
```

Output

```
Base Constructor: 100
Child Constructor
```

Conclusion

Parameterized constructors of parent classes must be invoked explicitly from child constructors using `super()` .

QUESTION 1(b)(ix)

? "The `equalsIgnoreCase()` and `compareToIgnoreCase()` methods differ by _____."

✓ Answer

🔥 "The `equalsIgnoreCase()` and `compareToIgnoreCase()` methods differ by their return type and comparison behavior."

☀ Detailed Explanation

Both methods are used for case-insensitive string comparison. However, they differ significantly in output type and intended purpose .

- `equalsIgnoreCase()`: Checks whether two strings are equal ignoring letter case. It returns a **boolean** value.
- `compareToIgnoreCase()`: Performs a lexicographical comparison ignoring case. It returns an **integer** value.

Code Example

```
class Main {
    public static void main(String args[]) {
        String s1 = "JAVA";
        String s2 = "java";
        System.out.println(s1.equalsIgnoreCase(s2)); // Returns true

        String s3 = "apple";
        String s4 = "banana";
        System.out.println(s3.compareToIgnoreCase(s4)); // Returns a negative
integer
    }
}
```

☀ Comparison Table

Feature	<code>equalsIgnoreCase()</code>	<code>compareToIgnoreCase()</code>
Return Type	<code>boolean</code>	<code>int</code>
Purpose	Equality check	Lexicographical comparison
Output	<code>true</code> / <code>false</code>	numeric (0, positive, negative)

Conclusion

Both methods perform case-insensitive comparison, but `equalsIgnoreCase()` checks equality while `compareToIgnoreCase()` determines relative alphabetical ordering .

QUESTION 1(b)(x)

? “Two strings can be concatenated by _____ or _____.”

✓ Answer

🔥 “Two strings can be concatenated by + operator or `concat()` method.”

🌟 Detailed Explanation

String concatenation means combining multiple strings into a single string. Java provides multiple mechanisms for this, but the most common are the + operator and the `concat()` method .

Code Example

```
class Main {  
    public static void main(String args[]) {  
        // Using + Operator  
        String s1 = "Object";  
        String s2 = "Oriented";  
        String s3 = s1 + s2;  
        System.out.println("Using +: " + s3);  
  
        // Using concat() Method  
        String s4 = "Java";  
        String s5 = "Programming";  
        String s6 = s4.concat(s5);  
        System.out.println("Using concat(): " + s6);  
    }  
}
```

🌟 Comparison Table

Feature	+ Operator	<code>concat()</code> Method
Type	Operator	Method

Feature	+ Operator	concat() Method
Null Handling	Converts automatically ("null")	May throw exception
Internal Optimization	Uses <code>StringBuilder</code>	Direct concatenation

Conclusion

Strings in Java can be concatenated using either the `+` operator or the `concat()` method  .